

Render

Cancel

Starting python3 kernel...

Welfare Analysis and Efficiency

Lecture 5

Div Bhagia

Render

Starting python3 kernel...

Introduction

```
Starting python3 kernel...
```

$$Q^D(p^*) = Q^S(p^*)$$

- **Long-run equilibrium:** Free entry $\rightarrow p = MC = \min\{AC\}$ (zero profits)
- **Welfare result:** Competitive equilibrium is efficient (maximizes $CS + PS$)

So far concerned with only one market i.e., **partial equilibrium** analysis.

Today: Move to **general equilibrium** and analyze more than one market simultaneously.

```
Starting python3 kernel...
```

- In the gas market: price rises, quantity falls, DWL
- But what about the car market? The labor market for oil workers? The market for public transit?

Changes in one market ripple through others. **General equilibrium** accounts for all these interconnections simultaneously.

Are competitive markets still efficient? Are there winners and losers?

Render

Starting python3 kernel...

Exchange Economy

Render

```
Starting python3 kernel...
```

- For now, just concerned with **efficient** allocation of existing goods, not how they are produced.
- Useful starting point to understand the main results
- We will then consider how production and factor markets fit into this framework

Render

```
Starting python3 kernel...
```

This is not the same as maximizing total welfare as we defined before.

Is there an equity implication of Pareto efficiency?

```
Starting python3 kernel...
```

- **Alberto** grabbed 5 pizza slices and 1 cupcake
- **Blake** grabbed 1 pizza slice and 3 cupcakes
- Total endowment: 6 pizza slices, 4 cupcakes

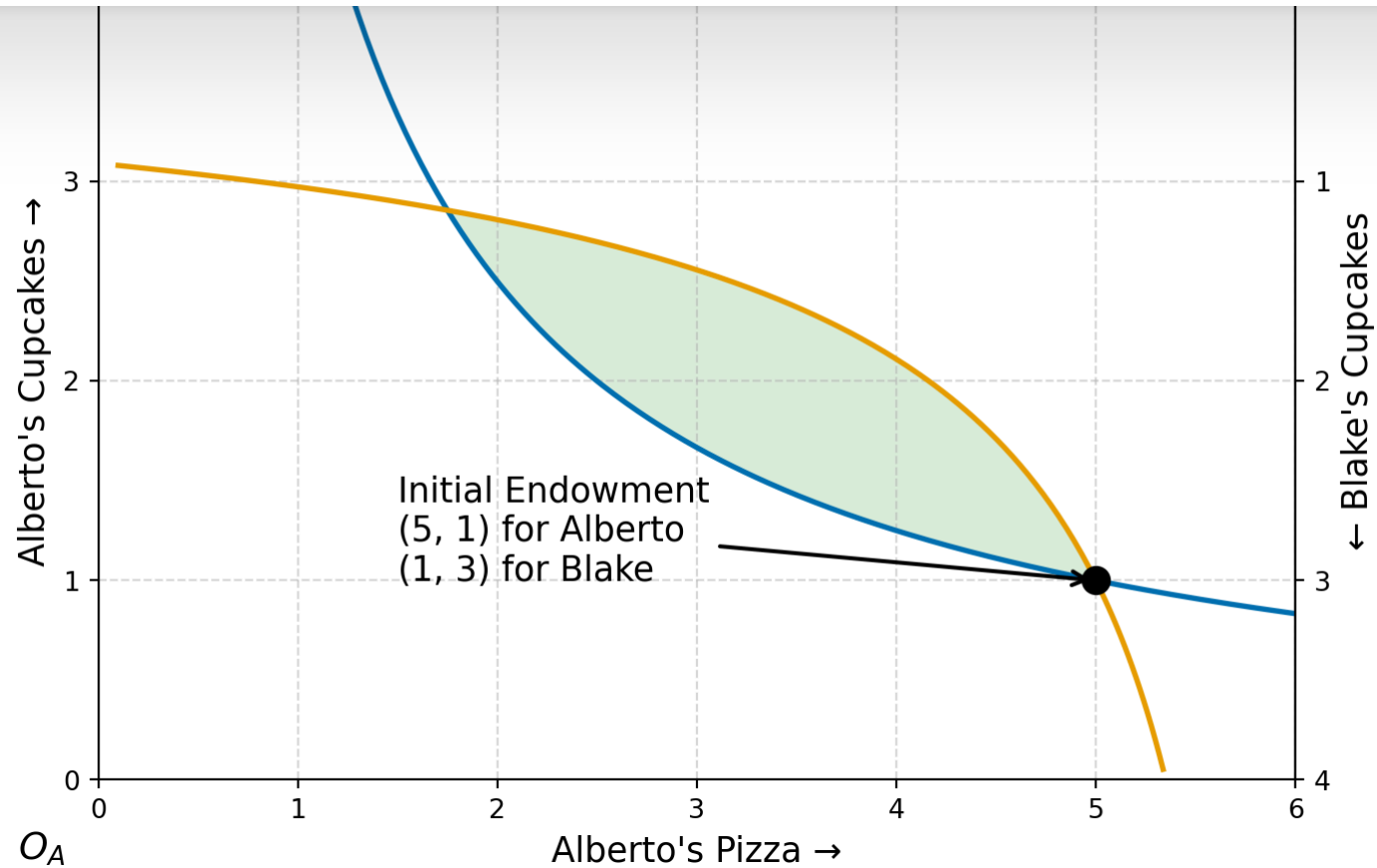
Can they do better by trading?

Depends on their preferences. Say:

- Alberto: $U_A = x^{0.5}y^{0.5}$ (prefers both equally)
- Blake: $U_B = x^{0.4}y^{0.6}$ (slightly prefers cupcakes)

Render

Starting python3 kernel...



Is the current allocation **Pareto efficient**?

```
Starting python3 kernel...
```

$$MRS_A = \frac{0.5}{0.5} \cdot \frac{y_A}{x_A} = \frac{1 \cdot 1}{1 \cdot 5} = \frac{1}{5}$$

Alberto is willing to give up 1/5 of a cupcake for one more slice of pizza.

Blake

$$MRS_B = \frac{0.4}{0.6} \cdot \frac{y_B}{x_B} = \frac{2}{3} \cdot \frac{3}{1} = 2$$

Blake is willing to give up 2 cupcakes for one more slice of pizza.

```
Starting python3 kernel...
```

- They can both be made better off if Alberto gives some pizza to Blake and Blake gives some cupcakes to Alberto.
- For an allocation to be **Pareto efficient**, we need $MRS_A = MRS_B$.
 - In this case, both of them value the last slice of pizza in terms of cupcakes equally, so there are no gains from further trade.
- The set of all allocations where $MRS_A = MRS_B$ is called the **contract curve** — the locus of Pareto efficient allocations.

Render

```
Starting python3 kernel...
```

$$\frac{0.5}{0.5} \cdot \frac{y_A}{x_A} = \frac{0.4}{0.6} \cdot \frac{y_B}{x_B}$$

Noting that $y_B = 4 - y_A$ and $x_B = 6 - x_A$, we can rewrite this as:

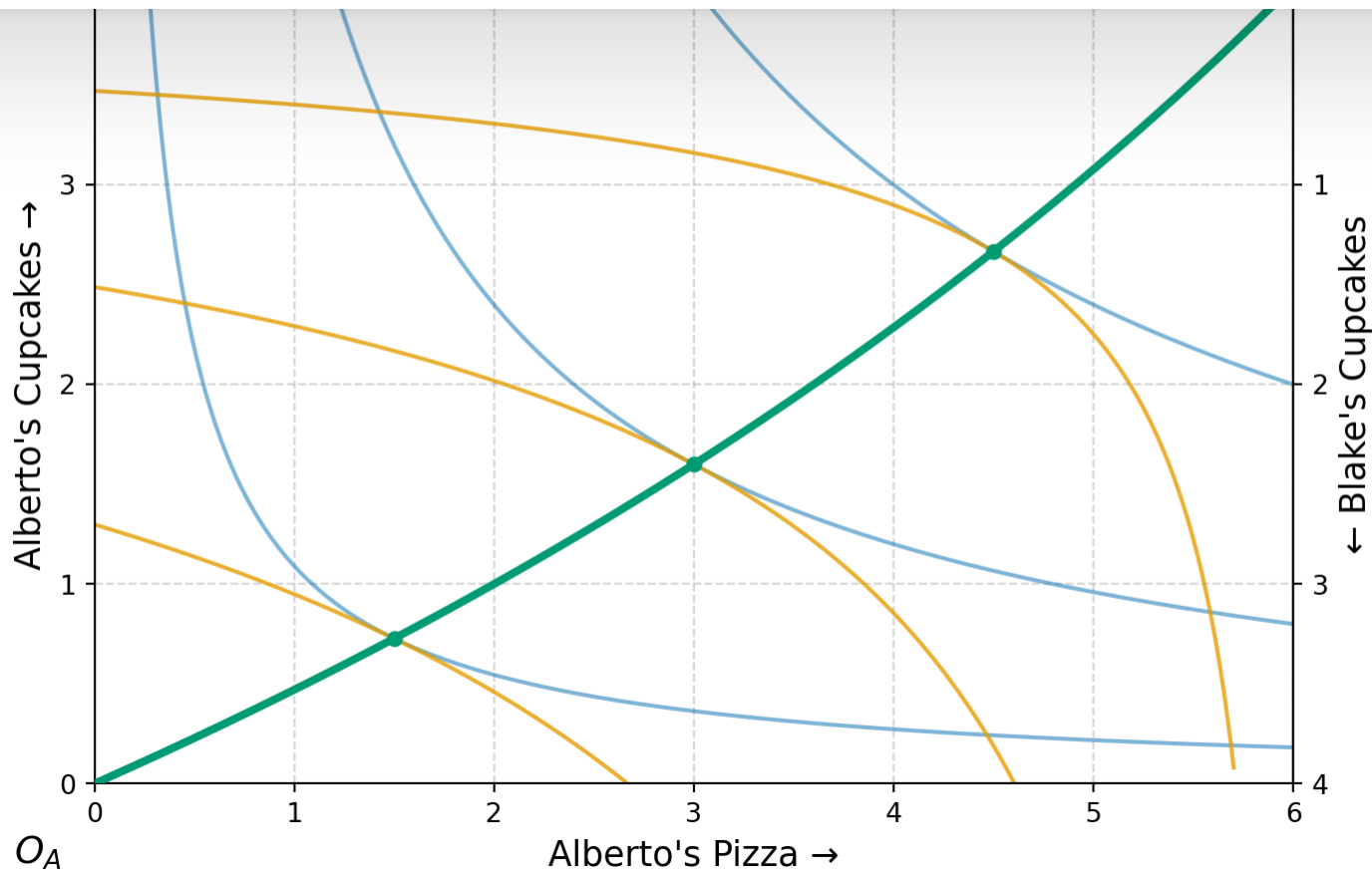
$$\frac{y_A}{x_A} = \frac{2}{3} \cdot \frac{4 - y_A}{6 - x_A}$$

Rearranging gives the equation of the contract curve:

$$3y_A(6 - x_A) = 2x_A(4 - y_A) \rightarrow y_A = \frac{8x_A}{18 - x_A}$$

Render

Starting python3 kernel...



What would Alberto and Blake end up choosing?

Render

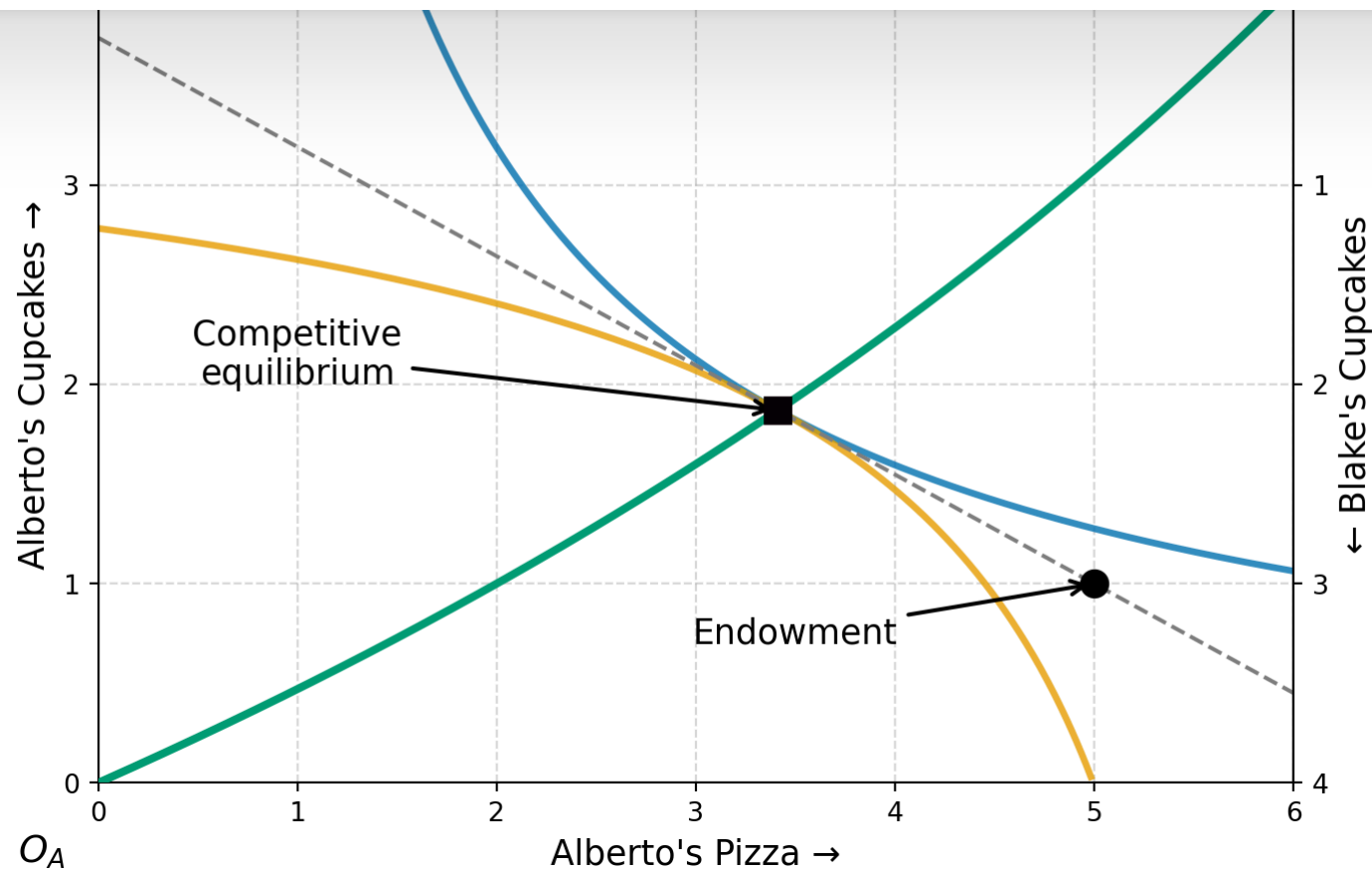
```
Starting python3 kernel...
```

- In a competitive market, prices will guide them to a particular point on the contract curve.
- Think there are many Alberto-like and Blake-like consumers who are all price-takers.

What should be the equilibrium prices?

Render

Starting python3 kernel...



Why does the budget line pass through the endowment point?

Render

```
Starting python3 kernel...
```

$$x_A = \frac{0.5(5p + 1)}{p} \quad y_A = 0.5(5p + 1)$$

$$x_B = \frac{0.4(p + 3)}{p} \quad y_B = 0.6(p + 3)$$

Market clearing for y : $y_A + y_B = 4$

$$0.5(5p + 1) + 0.6(p + 3) = 4 \rightarrow p \approx 0.55$$

Pizza is cheaper than cupcakes. Does that makes sense?


```
Starting python3 kernel...
```

- **Carleen** has endowment $(1, 0)$ and utility $U_C = x^{1/2}y^{1/2}$
- **Diego** has endowment $(0, 4)$ and utility $U_D = x^{1/2}y^{1/2}$

Normalize $p_y = 1$. Find:

1. Each person's demand functions for x and y
2. The equilibrium price p_x
3. The equilibrium allocation
4. Verify that markets clear

Is the equilibrium Pareto efficient? How do you know?

Render

```
Starting python3 kernel...
```

do the most wickedest of things for the greatest good of everyone.”

— John Maynard Keynes

Render

```
Starting python3 kernel...
```

Welfare Theorems

Render

```
Starting python3 kernel...
```

Local nonsatiation: For any bundle, there is always a nearby bundle the consumer strictly prefers. This just rules out “thick” indifference curves and ensures consumers spend their entire budget.

What it says: Markets with price-taking behavior and no externalities produce efficient outcomes. No one can be made better off without making someone else worse off.

What it does NOT say: Nothing about fairness, equity, or whether the outcome is socially desirable.

```
Starting python3 kernel...
```

- $\hat{x}_i \geq x_i^*$ for all i , with strict preference for some i

Step 1: If consumer i strictly prefers $\hat{x}_i$ to x_i^* , then $\hat{x}_i$ must cost more than i 's income (otherwise i would have chosen it):

$$p^* \cdot \hat{x}_i > p^* \cdot \omega_i$$

Step 2: If consumer j weakly prefers $\hat{x}_j$, local nonsatiation implies:

$$p^* \cdot \hat{x}_j \geq p^* \cdot \omega_j$$

Render

```
Starting python3 kernel...
```

$$p^* \cdot \sum_i \hat{x}_i > p^* \cdot \sum_i \omega_i$$

But feasibility requires $\sum_i \hat{x}_i \leq \sum_i \omega_i$. **Contradiction.** ■

Render

```
Starting python3 kernel...
```

- Works for any number of goods and any number of people
- Only needs local nonsatiation and price-taking behavior

Adam Smith's invisible hand, formalized: Decentralized decisions by self-interested agents, coordinated only by prices, produce an outcome that no central planner could improve upon (in the Pareto sense).

Render

```
Starting python3 kernel...
```

Naive approach: Price controls, rationing, direct mandates → typically create deadweight loss

Better question: Is there a way to use the power of markets (which we know produce efficient outcomes) to reach any desired efficient allocation?

This is exactly what the Second Welfare Theorem answers.

Render

```
Starting python3 kernel...
```

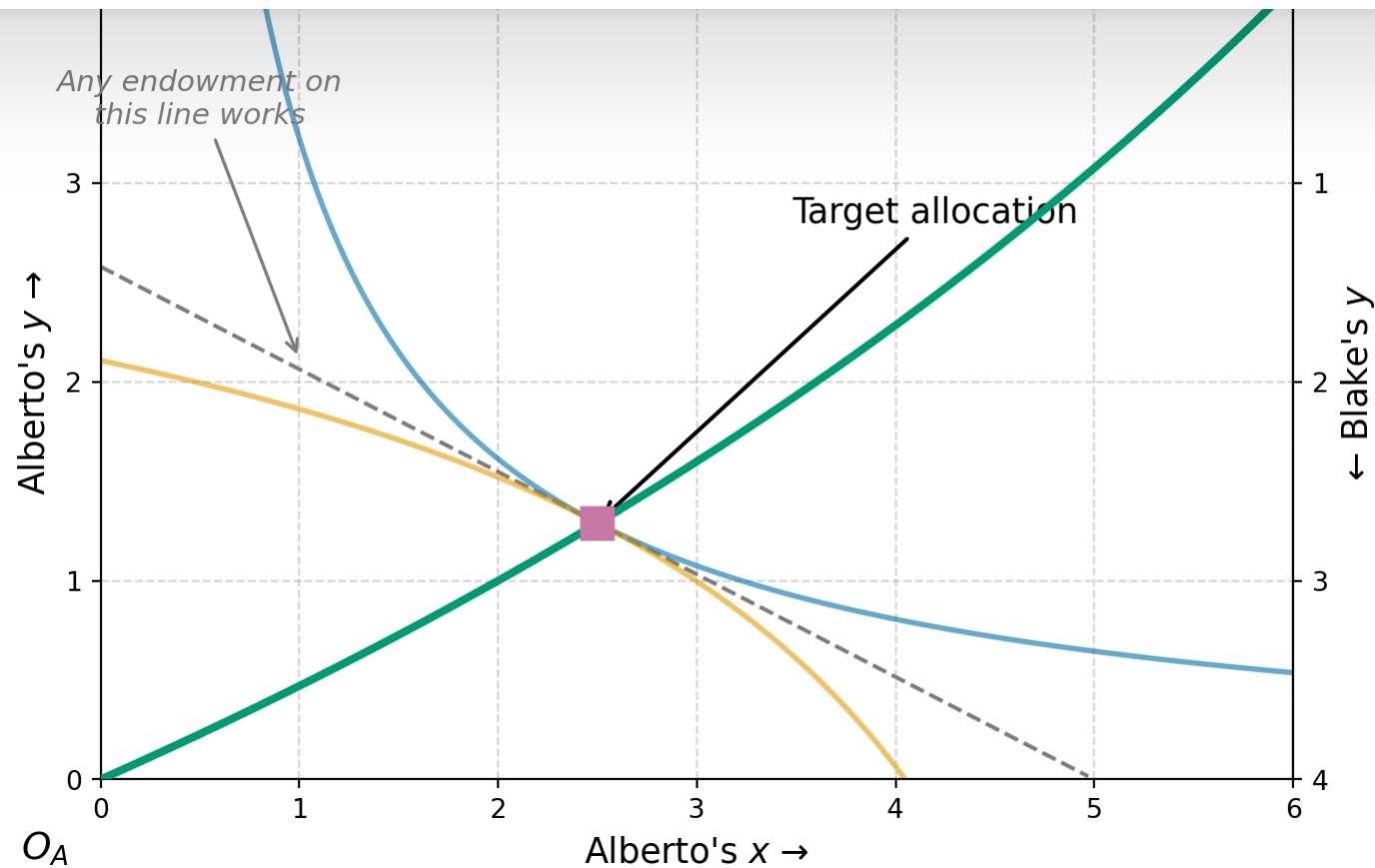
appropriate lump-sum transfers.

Translation: Pick any point on the contract curve. There exists a redistribution of endowments and a price vector such that x^* is the resulting competitive equilibrium.

Stronger assumptions than the First Theorem: Now we need convexity of preferences (diminishing MRS). Without convexity, the supporting price line may not be tangent to both indifference curves simultaneously.

Render

Starting python3 kernel...



Pick any Pareto efficient allocation. Redistribute endowments to a point on the supporting price line, then let markets work.

```
Starting python3 kernel...
```

resources.

Second Welfare Theorem: Government handles *equity* — use lump-sum transfers to adjust the starting point.

The policy recipe:

1. Redistribute endowments (lump-sum taxes and transfers)
2. Then let competitive markets do the rest

Why this is hard in practice: True lump-sum transfers are almost impossible. Any tax that depends on behavior (income, consumption, wealth) distorts incentives and creates deadweight loss. This tension between equity and efficiency is at the heart of public economics.

Render

Starting python3 kernel...

Social Welfare Functions

Render

```
Starting python3 kernel...
```

We need a way to aggregate individual wellbeing into a social ranking. Enter the **social welfare function**:

$$W = W(U_1, U_2, \dots, U_n)$$

This maps individual utilities into a single measure of social welfare. Different functional forms encode different ethical positions about inequality and the tradeoff between people's interests.

Render

```
Starting python3 kernel...
```

$$W = \sum_{i=1}^n U_i$$

Maximize total utility. Treats a dollar to a rich person and a dollar to a poor person equally.

Weighted utilitarian:

$$W = \sum_{i=1}^n a_i U_i, \quad a_i > 0$$

Society may value some individuals' utility more (e.g., higher weight on the worst-off).

Render

```
Starting python3 kernel...
```

$$W = \min\{U_1, U_2, \dots, U_n\}$$

Maximize the utility of the worst-off person. Extreme inequality aversion.

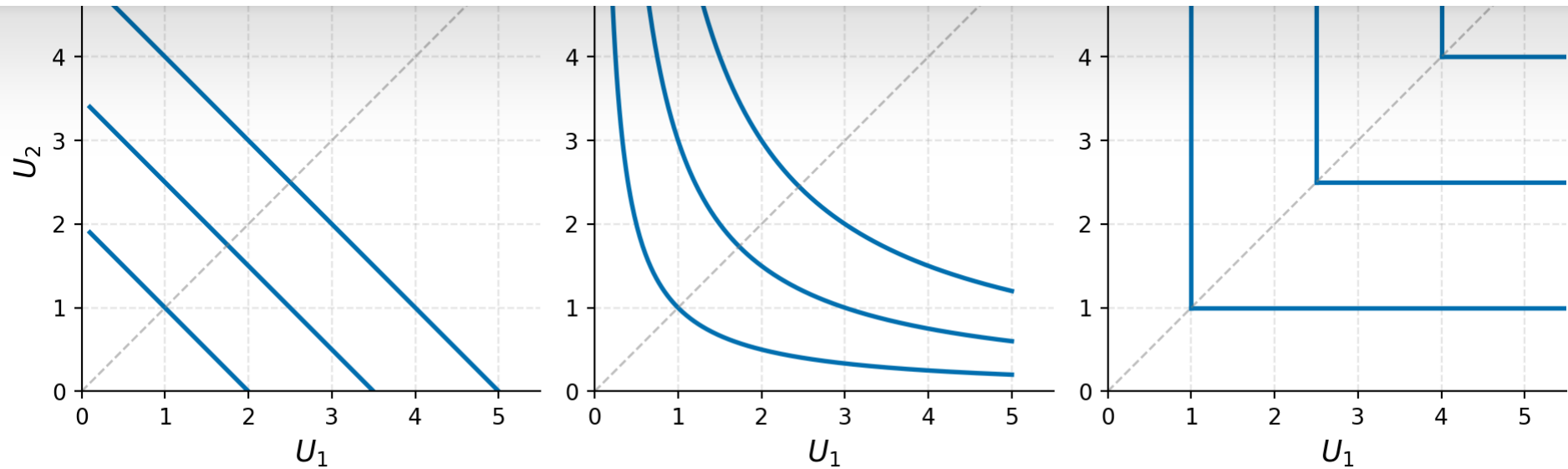
Nash (product):

$$W = \prod_{i=1}^n U_i$$

Equivalent to maximizing $\sum \ln U_i$. Intermediate inequality aversion.

Render

```
Starting python3 kernel...
```



The shape of social indifference curves reflects how much society is willing to trade off one person's utility for another's. Dashed line is the 45° line (perfect equality).

Render

```
Starting python3 kernel...
```

In the Edgeworth box, this means choosing the point on the **utility possibilities frontier** (derived from the contract curve) that reaches the highest social indifference curve.

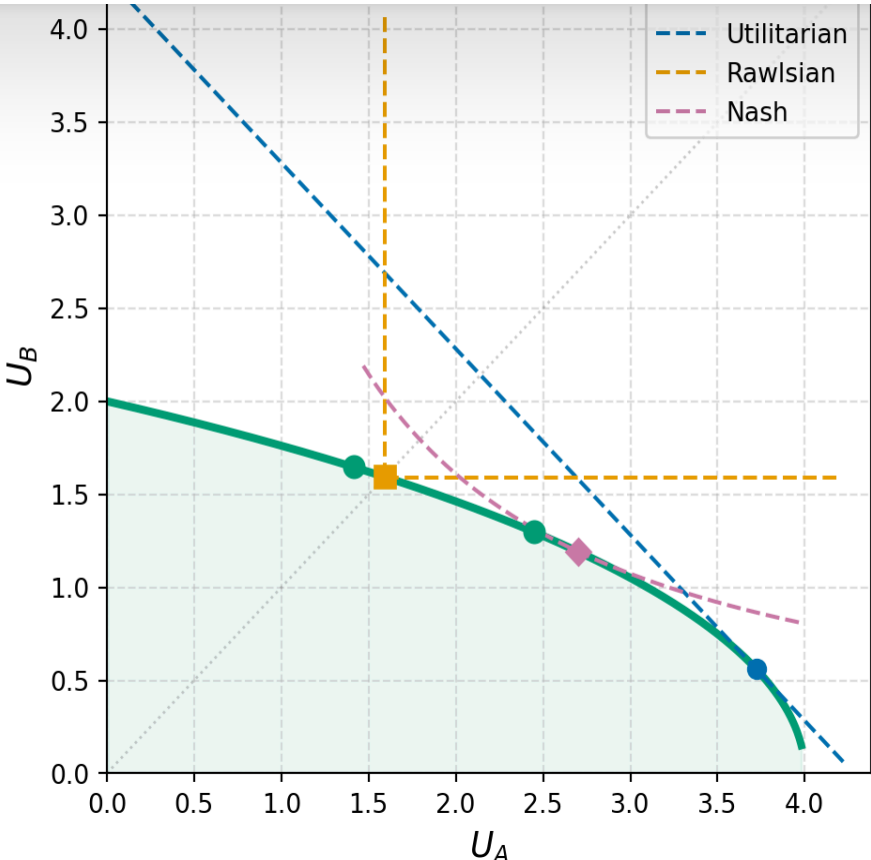
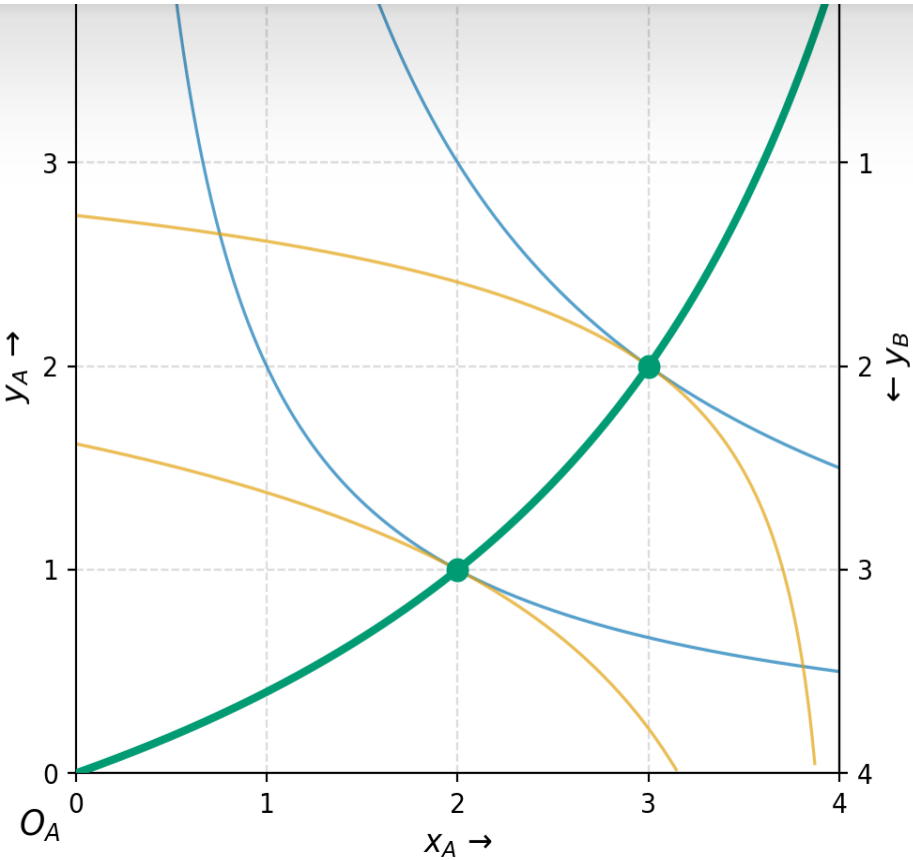
Different social welfare functions pick different points:

- **Utilitarian** → may tolerate significant inequality if total utility is higher
- **Rawlsian** → always picks the most equal feasible outcome
- **Nash** → something in between

Arrow's impossibility theorem (1951) tells us there is no “right” answer. Any SWF encodes value judgments that cannot be derived from individual preferences alone.

Render

Starting python3 kernel...



Render

```
Starting python3 kernel...
```

Adding Production

Render

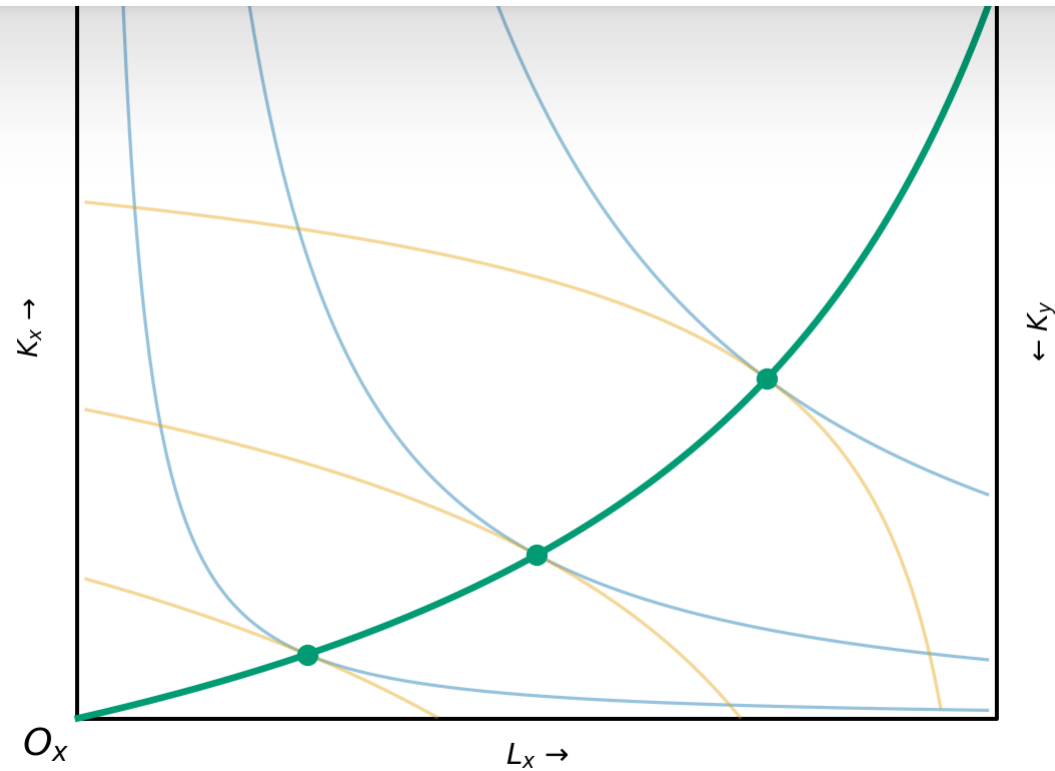
```
Starting python3 kernel...
```

Adding production introduces a new margin of efficiency: not just *how to allocate goods* (exchange efficiency), but also *how to allocate inputs across industries* (production efficiency) and *what mix of goods to produce* (product-mix efficiency).

The key new object is the **production possibilities frontier** (PPF).

Render

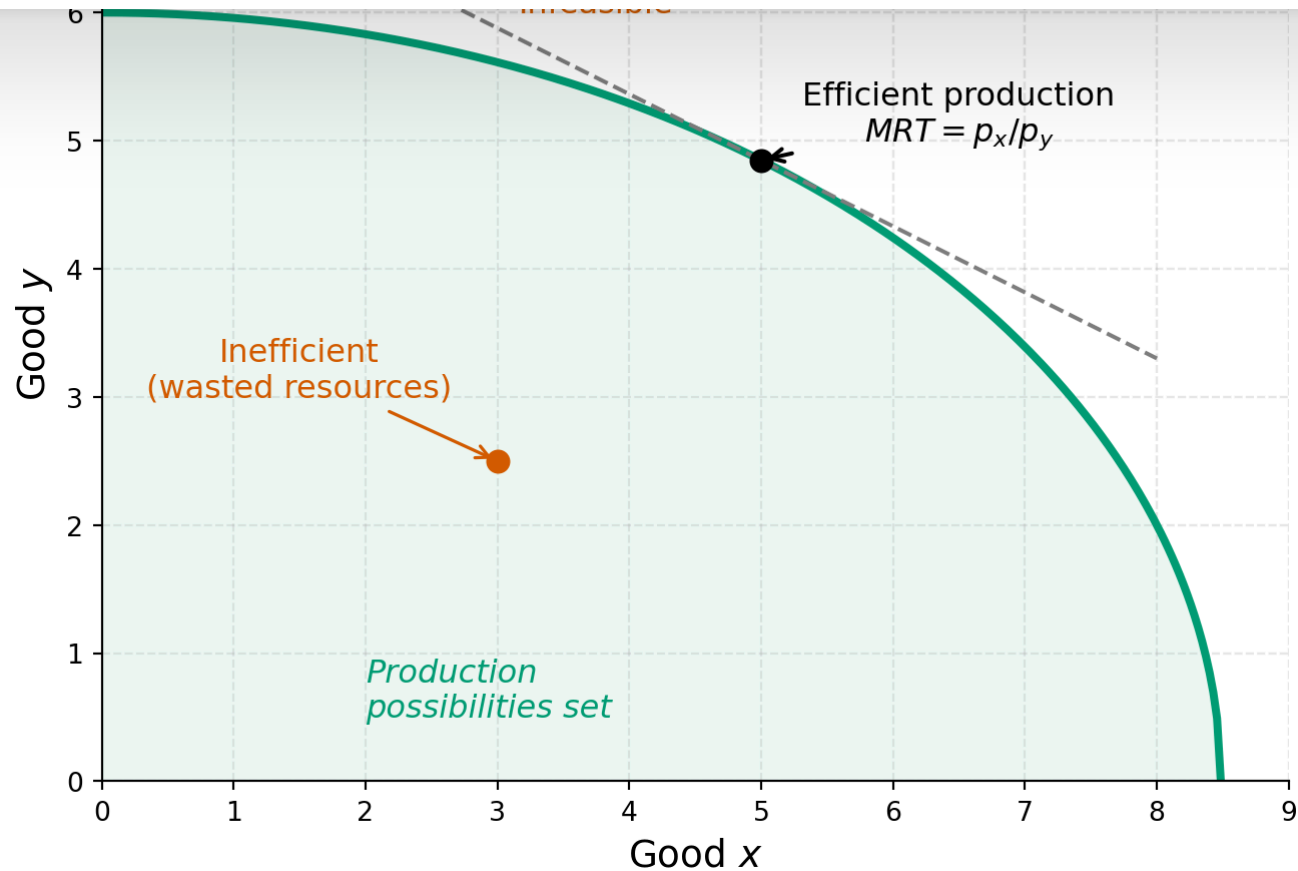
```
Starting python3 kernel...
```



Every point in the box is a full allocation of $\bar{L}$ and $\bar{K}$ between producing x and y . Efficient allocations lie on the **contract curve** where isoquants are tangent

Render

Starting python3 kernel...



The PPF shows all efficient combinations of output given the economy's resources and technology. Its slope is the **marginal rate of transformation**

```
Starting python3 kernel...
```

$$MRS_A = MRS_B$$

No gains from trading goods between consumers.

2. Production efficiency:

$$MRTS_x = MRTS_y$$

No gains from reallocating inputs between industries. This puts us *on* the PPF.

3. Product-mix efficiency:

$$MRS = MRT$$

The rate at which consumers are willing to trade x for y equals the rate at which the economy *can* transform x into y .

Render

Starting python3 kernel...

- Firms set $MRTS = w/r$ (same factor prices)
- Profit-maximizing firms produce where $MRT = p_x/p_y$